

Assembly Language Programming(ALP) 8086

Presented
by
M.Suma
kgrcet

Program 1: Increment an 8-bit number

- `MOV AL, 05H` Move 8-bit data to AL. Increment AL.
- `INC AL`

Program 2: Increment an 16-bit number

- `MOV AX, 0005H` Move 16-bit data to AX. Increment AX.
- `INC AX`

Program 3: Decrement an 8-bit number

- `MOV AL, 05H` Move 8-bit data to AL. Decrement AL.
- `DEC AL`

Program 4: Decrement an 16-bit number

- `MOV AX, 0005H` Move 16-bit data to AX. Decrement AX.
- `DEC AX`

Program 5: 1's complement of an 8-bit number.

- MOV AL, 05H Move 8-bit data to AL. Complement AL.
- NOT AL

Program 6: 1's complement of a 16-bit number.

- MOV AX, 0005H Move 16-bit data to AX. Complement AX.
- NOT AX

Program 7: 2's complement of an 8-bit number.

- `MOV AL, 05H` Move 8-bit data to AL. Complement AL.
- `NOT AL` Increment AL
- `INC AL`

Program 8: 2's complement of a 16-bit number.

- `MOV AX, 0005H` Move 16-bit data to AX. Complement AX.
- `NOT AX` Increment AX
- `INC AX`

Program 9: Add two 8-bit

numbers

```
MOV     AL,  
05H     MOV  
BL, 03H  ADD  
AL, BL
```

Move 1st 8-bit number to
AL. Move 2nd 8-bit number
to BL. Add BL with AL.

Program 10: Add two 16-bit

numbers

```
MOV     AX,  
0005H   MOV  
BX, 0003H  ADD  
AX, BX
```

Move 1st 16-bit number to
AX. Move 2nd 16-bit number
to BX. Add BX with AX.

Program 11: subtract two 8-bit numbers

```
MOV     AL,  
05H     MOV  
BL, 03H  SUB  
AL, BL
```

Move 1st 8-bit number to AL. Move 2nd 8-bit number to BL. **subtract** BL from AL.

Program 12: subtract two 16-bit numbers

```
MOV     AX,  
0005H   MOV  
BX, 0003H  SUB  
AX, BX
```

Move 1st 16-bit number to AX. Move 2nd 16-bit number to BX. **subtract** BX from AX.

Program 13: Multiply two 8-bit unsigned numbers.

MOV	AL,	Move 1 st 8-bit number to
04_H	MOV	Move AL. 8-bit number
BL, 02_H	MUL	Multiply BL with AL and the result
BL		will be in AX.

Program 14: Multiply two 8-bit signed numbers.

MOV	AL,	Move 1 st 8-bit number to
04_H	MOV	Move 2 nd AL. 8-bit number
BL, 02_H		Integer Multiply BL with AL and the
IMUL	BL	result will be in AX.

Program 15: Multiply two 16-bit unsigned numbers.

MOV	AX,	Move 1 st 16-bit number to
0004 _H	MOV	Move AL. 16-bit number
BX, 0002 _H	MUL	Multiply BX with AX and the result
BX		will be in DX:AX {4*2=0008=> 08=> AX , 00=> DX}

Program 16: Divide two 16-bit unsigned numbers.

MOV	AX,	Move 1 st 16-bit number to
0004 _H	MOV	Move 2 nd AL. 16-bit number
BX, 0002 _H	DIV	Divide BX from to AX. The result will be in AX & DX
BX		{4/2=0002=> 02=> AX , 00=>DX} (ie: Quotient => AX , Reminder => DX)

Detailed coding

16 BIT

PROGRAM	COMMENTS
MOV CX, 0000H	Initialize counter CX
MOV AX,[1200]	Get the first data in AX reg
MOV BX, [1202]	Get the second data in BX reg
ADD AX,BX	Add the contents of both the regs AX & BX
JNC L1	Check for carry
INC CX	If carry exists, increment the CX
L1 : MOV [1206],CX	Store the carry
MOV [1204], AX	Store the sum
HLT	Stop the program

Detailed coding

16 BIT

PROGRAM	COMMENTS
MOV CX, 0000H	Initialize counter CX
MOV AX,[1200]	Get the first data in AX reg
MOV BX, [1202]	Get the second data in BX reg
SUB AX,BX	Subtract the contents of BX from AX
JNC L1	Check for borrow
INC CX	If borrow exists, increment the CX
L1 : MOV [1206],CX	Store the borrow
MOV [1204], AX	Store the difference
HLT	Stop the program

16 BIT

MULTIPLICATION

PROGRAM	COMMENTS
MOV AX,[1200]	Get the first data
MOV BX, [1202]	Get the second data
MUL BX	Multiply both
MOV [1206],AX	Store the lower order product
MOV AX,DX	Copy the higher order product to AX
MOV [1208],AX	Store the higher order product
HLT	Stop the program

16 BIT

PROGRAM	COMMENTS
MOV AX,[1200]	Get the first data
MOV DX, [1202]	Get the second data
MOV BX, [1204]	Divide the dividend by divisor
DIV BX	Store the lower order product
MOV [1206],AX	Copy the higher order product to AX
MOV AX,DX	Store the higher order product
MOV [1208],AX	Stop the program
HLT	Get the first data

SUM of N-numbers

MOV

AX,0000

MOV SI,1100

MOV DX,1200

5 NUMBERS TO BE TAKEN as SUM

MOV

DX,0000

L1: ADD AX,
[SI] INC SI

INC DX CMP

CX,DX JNZ

L1

MOV [1200],AX HLT

Average of N-numbers

MOV

AX,0000

MOV SI,1100

MOV DX,1200

5 NUMBERS TO BE TAKEN
as AVERAGE

MOV

DX,0000

L1: ADD AX,
[SI] INC SI

INC DX CMP

CX,DX JNZ

L1

DIV CX

MOV [1200],AX HLT

AX=AX/5(AVERAGE OF 5
NUMBERS)

FACTORIAL of

MOV CX,0005

5

N

MOV

Factorial=5*4*3*2*1=120

DX,0000

MOV

AX,0001

L1: MUL CX

DEC DX

CMP CX,DX

JNZ L1

MOV [1200],AX HLT

ASCENDING ORDER

SORTING IN ASCENDING ORDER:

- Load the array count in two registers C_1 and C_2 .
- Get the first two numbers.
- Compare the numbers and exchange if necessary so that the two numbers are in ascending order.
- Decrement C_2 .
- Get the third number from the array and repeat the process until C_2 is 0.
- Decrement C_1 and repeat the process until C_1 is 0.

PROGRAM	COMMENTS
MOV SI,1200H	Initialize memory location for array size
MOV CL,[SI]	Number of comparisons in CL
L4 : MOV SI,1200H	Initialize memory location for array size
MOV DL,[SI]	Get the count in DL
INC SI	Go to next memory location
MOV AL,[SI]	Get the first data in AL
L3 : INC SI	Go to next memory location
MOV BL,[SI]	Get the second data in BL
CMP AL,BL	Compare two data's
JNB L1 	If $AL < BL$ go to L1
DEC SI	Else, Decrement the memory location
MOV [SI],AL	Store the smallest data
MOV AL,BL	Get the next data AL
JMP L2	Jump to L2
L1 : DEC SI	Decrement the memory location
MOV [SI],BL	Store the greatest data in memory location
L2 : INC SI	Go to next memory location
DEC DL	Decrement the count
JNZ L3	Jump to L3, if the count is not reached zero
MOV [SI],AL	Store data in memory location
DEC CL	Decrement the count
JNZ L4	Jump to L4, if the count is not reached zero
HLT	Stop

DESCENDING ORDER

SORTING IN DESCENDING ORDER:

- Load the array count in two registers C_1 and C_2 .
- Get the first two numbers.
- Compare the numbers and exchange if necessary so that the two numbers are in descending order.
- Decrement C_2 .
- Get the third number from the array and repeat the process until C_2 is 0.
- Decrement C_1 and repeat the process until C_1 is 0.

Note: change the coding **JNB** into **JB** in the **LINE**
L1 **L1** **10**

LARGEST, smallest NUMBER IN AN ARRAY

FINDING LARGEST NUMBER:

- Load the array count in a register C_1 .
- Get the first two numbers.
- Compare the numbers and exchange if the number is small.
- Get the third number from the array and repeat the process until C_1 is 0.

FINDING SMALLEST NUMBER:

- Load the array count in a register C_1 .
- Get the first two numbers.
- Compare the numbers and exchange if the number is large.
- Get the third number from the array and repeat the process until C_1 is 0.

LARGEST NUMBER

PROGRAM	COMMENTS
MOV SI,1200H	Initialize array size
MOV CL,[SI]	Initialize the count
INC SI	Go to next memory location
MOV AL,[SI]	Move the first data in AL
DEC CL	Reduce the count
L2 : INC SI	Move the SI pointer to next data
CMP AL,[SI]	Compare two data's
JNB L1	If $AL > [SI]$ then go to L1 (no swap)
MOV AL,[SI]	Else move the large number to AL
L1 : DEC CL	Decrement the count
JNZ L2	If count is not zero go to L2
MOV DI,1300H	Initialize DI with 1300H
MOV [DI],AL	Else store the biggest number in 1300 location
HLT	Stop

SMALLEST
NI IMPED

PROGRAM	COMMENTS
MOV SI,1200H	Initialize array size
MOV CL,[SI]	Initialize the count
INC SI	Go to next memory location
MOV AL,[SI]	Move the first data in AL
DEC CL	Reduce the count
L2 : INC SI	Move the SI pointer to next data
CMP AL,[SI]	Compare two data's
JB L1	If $AL < [SI]$ then go to L1 (no swap)
MOV AL,[SI]	Else move the large number to AL
L1 : DEC CL	Decrement the count

Byte Manipulation(Using Logical Instructions){ex:

Example 1:

```
MOV AX,  
[1000] MOV  
BX,[1002]  
AND AX,BX  
MOV  
[2000],AX HLT
```

Example 2:

```
MOV AX,  
[1000] MOV  
BX,[1002] OR  
AX,BX MOV  
[2000],AX HLT
```

Example 3:

```
MOV AX,[1000]  
MOV BX,  
[1002] XOR  
AX,BX MOV  
[2000],AX HLT
```

Example 4:

```
MOV AX,[1000]  
NOT AX  
MOV [2000],AX  
HLT
```

STRING

1. Copying a String (MOV)

SB) MOV CX,0003 copy 3 memory locations

MOV SI,1000

MOV DI,2000

L1 CLD

MOV decrement

SB DEC CX

CX JNZ

2. Find &

PROGRAM	COMMENTS
MOV DI,1300H	Initialize destination address
MOV SI,1400H	Initialize starting address
MOV CX, 0006H	Initialize array size
CLD	Clear direction flag
MOV AL, 08H	Store the string to be searched
MOV BH,30H	Store the string to be replaced
REPNE SCASB	Scan until the string is found
DEC DI	Decrement the destination address
MOV BL,[DI]	Store the contents into BL reg
MOV [SI],BL	Store content of BL in source address
MOV [DI],BH	Replace the string
HLT	Stop

Procedure s

- **Procedure** is a part of code that can be called from your program in order to make some **specific task**. Procedures make program more **structural and easier to understand**.
- syntax for procedure declaration:

```

name PROC..... ; here goes the
          .... code
          RET ..... ; of the
name ENDP..... procedure ...

```

here **PROC** is the procedure name.(used in top & bottom) **RET** - used to return from OS. **CALL**-call a procedure **PROC & ENDP** – compiler directives **CALL & RET** - instructions

EXAMPLE 1 (call a procedure)

ORG 100h

CALL m1

MOV AX,

2

RET ;

m1 PROC

return to
~~MOV BX,~~
operating
~~5 RET ;~~
system.
m1

return to
caller.

ENDP

- ~~END~~ above example calls procedure **m1**, does
MOV BX, 5 & returns to the next
instruction after ~~CALL~~ **MOV AX, 2**.

Example 2 : several ways to pass parameters to procedure

```
ORG 100h
MOV AL, 1
MOV BL,
2 CALL
m2
CALL m2
CALL
m2
CALL
m2
RET PROC
MUL BL
; return to
operating
system.
ENDP
```

; $AX = AL * BL$.

; return to
caller.

value of AL register is update every time
the procedure is called.

final result in AX register is **16 (or 10h)**

- Stack is an area of memory for keeping **temporary data**.
- STACK is used by **CALL & RET** instructions. **PUSH** -stores 16 bit value in the stack. **POP** -gets 16 bit value from the stack.
- PUSH and POP instruction are especially useful because we don't have too much registers to operate
 1. Store original value of the register in stack (using PUSH).
 2. Use the register for any purpose.
 3. Restore the original value of the register from stack (using POP).

Example-1 (store value in STACK using PUSH & POP)

ORG 100h

MOV AX,

1234h

PUSH AX

; store value of AX in stack.

MOV AX,

5678h

POP

; restore the original value of AX.

AX RET

END

Example 2: use of the stack is for exchanging the values

```
ORG
MOV      AX,      ; store 1212h in AX.
1212h      MOV      ; store 3434h in BX
BX, 3434h   ; store value of AX in
PUSH AX     stack.
PUSH BX     ; store value of BX in
POP         stack.
AX          ; set AX to original value of BX.
POPBX       ; set BX to original value of AX.
RET
END
```

push 1212h and then 3434h, on pop we will first get 3434h and only after it 1212h

MACRO

S

- Macros are just like procedures, but not really.
- Macros exist **only** until your code is **compiled**
- **After compilation** all macros are **replaced** with **real instructions**
- several macros to make coding easier(**Reduce large & complex programs**)

Example (Macro definition)

```
name MACRO [parameters,...]  
<instructions  
> ENDM
```

Example1 : Macro

Definitions

SAVE

MACRO

PUSH

AX

PUSH

BX

PUSH

MACRO

ENDM

CX POP

BX POP

AX

ENDM

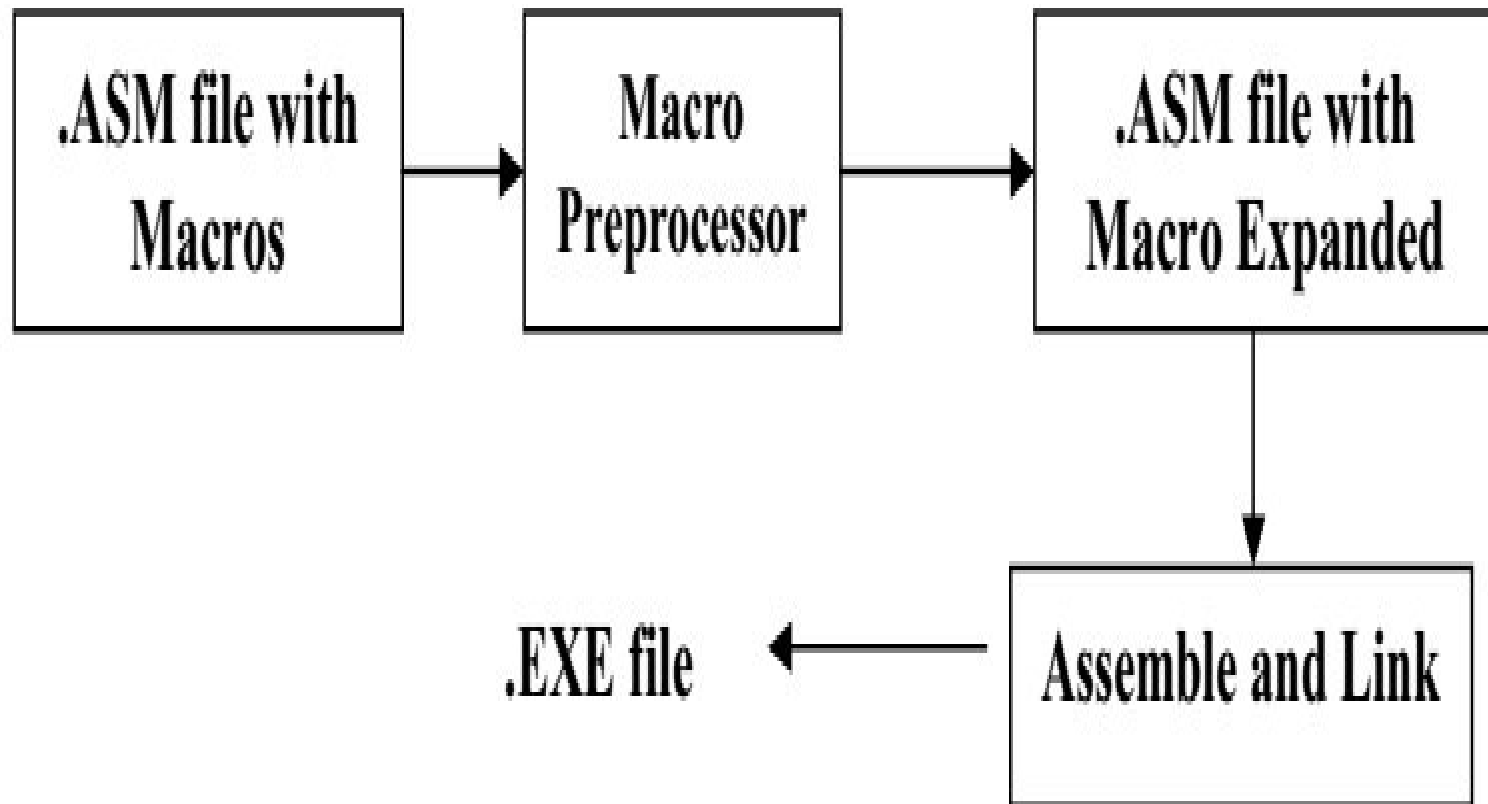
definition of MACRO name

SAVE

Another definition of MACRO name

RETRIEVE

RETRIEVE



MACROS with Parameters

Example:

COPY

MACRO

x, y

; macro named **COPY**
with 2 parameters{**x**,
y}

PUSH

AX MOV

AX, **x**

MOV